A COMPARATIVE STUDY OF THE PERFORMANCE OF LSTM IN FOREX AND STOCK MARKETS TIME SERIES FORECASTS

Rudy C. Yuen[1]
June 2020

Abstract

Traditional papers using machine learning or statistical algorithms to predict markets would use stock market datasets due to the limited level of randomness existing within the dataset, providing a higher certainty for algorithms chosen for the particular study to capture statistical features within the dataset. In this paper, this investigation of market predictions has been extended to the Foreign Exchange Market with the better algorithm in distinct comparative papers, Long Short-Term Memory (LSTM).

The comparative study evaluates whether if the complexity to predict the two markets with hypothetically different complexity will result in a higher error metric, using a univariate Gaussian Distribution hypothesis test and a Kullbeck-Leiber Divergence. To provide more opportunities for the LSTM to capture statistical features within the datasets, I have added linear layers before and after the LSTM hidden layers to filter out the white noise. Finally, the neural network is trained using the datasets, and the results are presented.

The results for this comparative study show that there is insufficient evidence to show that there is a higher difficulty to predict 6 major Foreign Exchange Markets than 5 of the many most-traded stocks even though the root mean squared error metric between the Foreign Exchange Market prediction is higher than the stock market prediction. Given the conclusion, the conclusion that the Foreign Exchange Market and the stock market has a similar difficulty to predict using a Long Short-Term Memory Neural Network Architecture is drawn.

Keywords: Long Short-Term Memory, Financial Series Time Series Forecasting, Foreign Exchange Markets, Technical Analysis, Recurrent Neural Networks, Artificial Neural Networks

[1] Department of Computer Science, Warwick School, Warwick Independent Schools Foundation, Warwick, CV34 6PP, United Kingdom

WRITER DECLARATION

Declaration of Originality

I, Yuen Cheuk-wang Rudy of Warwick School, hereby declare that this paper and the work described in it, unless referenced, are my work. I also declare that this paper, to my best knowledge, does not contain material that has already been used for any substantial extent for a comparable purpose.

This paper is a result of my work and is not a result of any work done in collaboration. I also confirm that I have not manipulated any data obtained through the programs used in this paper.

I: INTRODUCTION

Forecasting anything has been one of the most desired abilities since the history of time. In the recent 50 years, the stock and the Forex market has become a predominant pathway to create a living internationally. Equipped with the ability to forecast in time, investors can put their money in particular markets which will eventually create a profit. The concept of forecasting in time has been widely utilized when machine learning algorithms were created.

In 1997, Hochreiter and Schmidhuber (Hochreiter and Schmidhuber, 1997) introduced LSTM, Long Short-Term Memory in attempt to solve Exploding and Vanishing Gradients and short-term dependencies within Recurrent Neural Networks at that time. This new piece of algorithm broke accuracy records for time series forecasting within all neural networks. Up to now, this neural network algorithm is still widely used within the industry and research work for different purposes to do with signal processing, such as Natural Language Processing (NLP) and Stock Market Predictions, or combined with other neural network architectures, such as Convolutional Neural Networks (CNNs), in tasks such as live video captioning and biomedical purposes.

Recent papers which are related to stock market predictions such as one which compares Autoregressive Integrated Moving Average (ARIMA) and LSTM (Sima, *et al.*, 2018) and another which compares between LSTM and Deep Neural Networks (DNN) (Shah, *et al.*, 2018) has drawn to a similar conclusion that LSTM predicts the stock market better than the other algorithm. This coincide conclusion provokes a research interest to LSTM: given that the Forex market is more stochastic than the stock market, can the LSTM neural network architecture predict the Forex market as well as the stock market?

Section 1: An Economical Introduction

The 'Efficient Market Hypothesis' (EMH) is an economical hypothesis stated by the Nobel Prize in Economic Sciences Laureate Dr Eugene Fama in his economics doctorate dissertation. The concept of efficient markets is stating that the current state of the market is a result of the incorporation of all information relevant and open. This means that it is impossible to 'defeat' the market since it shows a true reflection of the price which the market is worth, hence the only way to create a mass profit within a short period is to invest in high-risk markets. (Chappelow, 2019)

Yet, there are a lot of FinTech researchers and economist who are actively seeking mathematical or computational algorithms which would be able to act as an alpha-generating machine. In recent years, high-frequency trading has been introduced as a way for companies and hedge funds to generate a small profit over fractions of a second. However, for both methods, it is very complex for models to consistently generate a profit since the stock market has a lot of white noise and irrational events.

It remains a controversial argument of whether EMH is valid for all markets nowadays surrounded by technological devices, some argue that the EMH should stand stronger, since everyone can get access to similar information almost at the same time, causing the stock market to reflect immediately. However, in a similar argument, insider

trading, artificial intelligence, mathematical and computational models can be applied at a larger scale which would defy the EMH.

No matter whether or not the EMH is a true representation of the market's behaviour, the market trend is very stochastic, irrational and occasionally, illogical.

There exist the two main types of markets, the stock market and the Forex market. They have different degrees of randomness due to their nature. Depending on the type of stock and the nature of the company, different stocks can perform well at different economical states of the country, for example, some might do well when the country's economic status is on a deflation. However, the Forex market is an indicator of more items, such as the country's economic status, the country's current state and a reflection on all other factors such as the foreign reserves.

Investments in the Forex Market is often considered as not profitable by investors since the Forex Market should not have large fluctuations over time as there is a government behind backing the currency up and arbitrage corrected. However, certain activities targeted at the Forex Market, such as short selling, where an investor deliberately sell a lot of the country's money to create a huge profit such as George Soros who sold billions of pounds which caused the Black Wednesday crisis. Some other events such as the global financial crisis in 2007 or Brexit would also cause the price to drop at a large extent. These activities in the market have adversely affected the price of the stock which would deem to be unpredictable by a lot of investors in the world. However, these activities might not be reflected accordingly in a stock's price.

Section 2: Neural Networks

Neural Networks are inspired by the human brain which contains neurons and interconnected by synapses. In a computational neural network, there are layers of neurons and they are interconnected by weighted channels.

There are two parameters and a function which exist in a computational neuron: a weight and a bias along with an activation function. During initialization, the weight and the bias are set to be a matrix consisting of random numbers usually ranging between 0 and 1 and 0 respectively. When data is being propagated through the network, they go through a linear transformation with the bias and the weight; where the weight acts as the gradient of the linear transformation and the bias as the intercept. The values get passed into the activation function. The function would return a value, which the larger the value returned, the more the neural network deem the data important.

Since during the initialization, the neural network is made up of a random value, the first time the data is propagated into the network would result in a solution which is not desirable. The value would then be compared to the desired value using a function named the loss function. The loss would then be backpropagated into the network using partial differential chain rules to reduce the loss when the same set of data is propagated the next time.

If each time the dataset is passed into the neural network, a process named 'epochs' generates a loss, which is denoted $L(x)$, and the value of n is the number of epochs and θ is the neural network's current state such that the neural network's state at a particular epoch is θ_n, it is expected that $L(\theta_1) > L(\theta_2) > \cdots > L(\theta_n)$, such that

$\lim_{n\to\infty} L(\theta_n) = 0$ and we are trying to seek $\theta = \underset{\theta}{\arg min} L(\theta)$. This is sought through a process called 'gradient descent'. This process is done using some numerical methods such as the Newton Raphson Method, $x_{n+1} = x_n - \frac{f(x)}{f'(x)}$, or others such as stochastic gradient descent. The general concept of gradient descent is that $\theta_{n+1} = \theta_n - \eta \frac{\partial L(\theta_n)}{\partial \theta}$, where the value η is the learning rate.

For each parameter in a neural network, there is a process of gradient descent to seek the best value of it to generate an overall loss which is minimal. A metaphor for this process is that there is a microscope where there are a lot of different knobs to control the magnification and lens' clarity. The very first step to creating the best image is to turn one knob and stop until the image is the clearest for that particular knob. Then continue this process for until the last knob is on a setting which provides the clearest setting for the particular setting on the other knobs, then repeat this process from the first knob on, where the epoch would now be 2.

Using this method, eventually, there would be a very clear image, it might not be the best settings for the particular sample observed under the microscope, but this method provides a good enough image through a heuristic process.

Similarly for neural networks, there are hundreds of thousands of parameters in an entire neural network. It is very difficult for neural networks to reach a global minimum of loss values in a realistic situation. Neural networks, on the other hand, would reach a local minimum, which might provide a prediction good enough for the case, but not necessarily the best.

The process of pursuing the local minimum, not only using gradient descent but will also use optimizers so speed up the process. Commonly used optimizers include stochastic gradient descent (SGD) and Adam (Kingma and Lei Ba, 2015). A common parameter within the two optimizers includes learning rate, which is the η mentioned above.

It is vital to choose a suitable learning rate for optimizing operations. In the simplest case, where the dataset is perfectly normalized and the graph for the loss is an inverted dome. Assuming that the learning rate is overly large, the current loss would slosh between the curvatures in the dome, causing the process of learning to be ineffective and inefficient. However, if the learning rate is too small, the process of gradient descent would take too much time. It is crucial to reduce the learning rate from the start of the process, causing the state of the neural model to converge to a minimum.

Section 3: Recurrent Neural Networks and LSTM

The concept of recurrent neural networks is that every recurrent cell stores a cell state which serves a memory, which does not exist within ordinary neurons such as Convolutional Neurons, or Ordinary Linear Neurons. This memory would allow the network to remember key features and interrelate it with similar features in the future, this function is named as 'dependency'.

In sequence processing tasks where recurrent neural networks are required, such as NLP, it is common that the very first words in a sentence can be affecting the last word. For example to fill the blank in the sentence 'When I purchased my Apple Laptop has the

operation system ___ installed.', the 5th word 'Apple', is very dependent to which word should be placed in the blank (the 11th word). However, in recurrent neural networks, it is very hard for the network to predict the word 'MacOS' since the word 'Apple', the only word that relates to 'MacOS', are far away in the sentence (6 words apart in a 12 worded sentence). This property of ordinary recurrent neural networks is called short-term dependencies.

In ordinary datasets, it is very often that the dataset consists of thousands of data points. When the dataset is propagated through a recurrent neural network, a multiplication operation is performed to it. This means that the hidden state is raised to the power of the number of data points. If the hidden state is a number less than 1, the hidden state would converge to 0, for example $0.99^{1000} \approx 4.32 \times 10^{-5}$, whereas if the hidden state is larger than one, the hidden state would diverge to infinity, *i.e.* $1.01^{1000} \approx 2.2 \times 10^{4}$. Either case that the 'memory' converges to 0 or diverges to infinity, this signifies that the memory would no longer be useful within the neural network's calculation. This problem is also named as the vanishing and exploding gradient problem.

These two major problems, short-term dependencies and vanishing and exploding gradients are two major reasons for the downfall of recurrent neural networks until the concept of Long Short-Term Memory (LSTM), by Hochreiter and Schmidhuber, is published in 1997, Neural Computation Issue 8.

The LSTM neuron contains three different parts named gates. They are the input gate, forget gate and output gate. The main concept of LSTM is that the cell automatically filters out some useless information and takes in the most useful information for analysis through a forget procedure, causing the hidden state to consistently stay within the range of 0 and 1 at a reasonable level.

There are 6 main equations associated with an LSTM recurrent cell, enlisted below:

$$f_{t=} = \sigma(W_f[x_t, h_{(t-1)}] + b_f) \tag{1}$$

$$\overline{c_t} = \tanh(W_c[x_t, h_{(t-1)}] + b_c) \tag{2}$$

$$i_t = \sigma(W_i[x_t, W_i h_{(t-1)}] + b_i) \tag{3}$$

$$o_t = \sigma\big(W_o[x_t, h_{(t-1)}] + b_o\big) \tag{4}$$

$$h_t = o_t * \tanh c_t \tag{5}$$

$$c_t = f_t * c_{(t-1)} + i_t * \overline{c_t} \tag{6}$$

Equation *1* denotes how much of the previous information to forget, since the function is placed in a sigmoid function, the range of the output is within 0 and 1. Reviewing equation *6* which controls the output of a particular cell iteration and cell state, the output of Equation *1* controls how much the previous cell state to discard.

Equation *2* and *3* controls how much of the information newly received should be propagated into the next cell state; Equation *4* controls how much of the information should be passed on to the hidden state, and Equation *5* controls how much of the information should be passed out of the cell, to the cell's next time period as its hidden state.

Equation *6* controls how much of the information passed into the cell at a particular time period and how much information from the previous time period should be passed to the next time period.

The graphical illustration of LSTM cell and its functionality is illustrated as Figure 1.

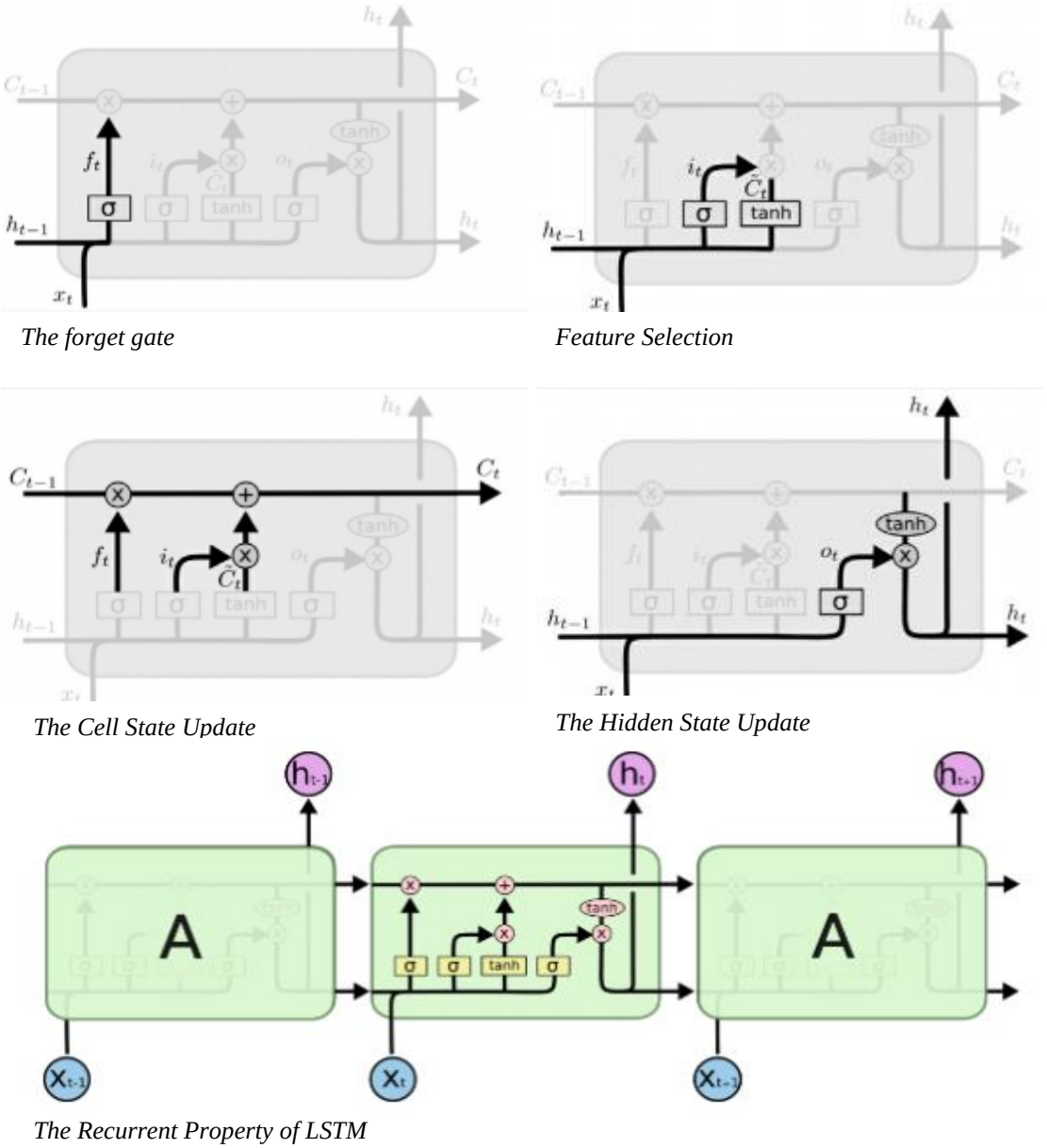

Figure 1: The functionality of an LSTM Cell

Equipped with the ability to regulate the hidden state's value, and discard of information that is not required, LSTM can solve the vanishing or exploding gradient problem, and they are very effective to establish long term-dependencies, to relate information from the past to future events, such as the sentence used above, quoted here: 'When I purchased my Apple Laptop has the operation system ___ installed.'

Using an LSTM neural network trained to process natural languages, the network would be able to relate the word 'Apple' to 'MacOS' after proper training with suitable epochs.

Section 4: Hyperparameters in Neural Networks

As introduced in *'Section 2: Neural Networks'*, there are processes where parameters are adjusted within a neural network, such as learning rate, weights and biases which are to assist, or directly affect the process of reducing the loss into a local or global minimum. There are also parameters within neural networks which directly affect the neural network's loss, however, are not changed throughout forward or backward propagation. These constants within neural networks are called hyperparameters.

These constants would usually affect the performance of the neural network, such as the time taken, whether the model underfits or overfits the data, the efficiency of the network, *et cetera*. Constants as such include epochs, batch size, the number of neurons in a layer, the number of hidden layers, the number of outputs required, *et cetera*, are all features of hyperparameters. Their usages and definitions are listed below:

1. Epochs: Epochs are the number of times the complete dataset is propagated through in the neural network. Using a higher epoch will result into the training loss to approach zero, however, overfitting might occur, whilst using too few epochs will result into underfitting which means the statistical representations of the network are not fully captured.
2. Batch Size: Each batch is the entire dataset sliced up into packets of data, such that the packets get propagated each time rather than the entire dataset, this increase the accuracy and the efficiency of the network. The batch size should be determined by the amount of data existing within the dataset and depending on the neural network algorithm chosen.
3. The number of neurons in a layer: By increasing the number of neurons per layer, more statistical representations can be captured, however, if there are more than sufficient neurons overfitting will very quickly occur, whilst if there are too few neurons, there might not be sufficient chances for the statistical representations to be learnt.
4. The number of hidden layers: The deeper the network, the more features within the dataset is captured. It is very common for neural networks to have very large hidden layers at the first and smaller layers at the end. This allows the more significant features to be remembered to a larger extent and more insignificant features to be remembered as a smaller extent.
5. The number of outputs required: There is an expected amount of outputs in every neural network, for example in a classification task, there might be a fixed number of categories that a convolutional neural network is required to output, such as the CIFAR10 and the MNIST dataset would require 10 linear neurons at the output layer since both of them expect 10 different answers. For example in this case of a time series prediction, the number of output layer neurons required would determine how many data points are required to be predicted.

These constants would inevitably improve the network's performance if tuned the correct way. To complete the process of hyperparameters tuning, linear methods, grid methods, random searches, *et cetera* are commonly used to seek the best values for each of these hyperparameters since most tasks would require different hyperparameter values. There are also neural network algorithms constructed on a professional basis and are tested rigorously, such as ResNet18 and VGGNet23 which are trained and constructed for tasks such as classification and would have a high performance for these tasks.

II: RELATED WORK

As computational technologies develop, more and more papers regarding forecasting time series using different methods such as statistical and computational methods are published. With the development of computational techniques and computational power, more and more algorithms are created to forecast stocks.

In this chapter, two comparative studies' conclusion on Long Short-Term Memory Recurrent Neural Networks will be used quoted.

Shah et al., 2018 has used two neural networks, a Long-Short Term Memory Network and a Deep Neural Network. In this paper, an LSTM model with 4 layers, where the first layer is constructed by the amount of data required to create an output, which they used 50 data points to create an output. The second layer is constructed with 50 LSTM hidden units, the third layer is another linear layer with 100 neurons, whereas the final layer is dependent on the size of the output. It is compiled with the ADAM optimizer and there is a 25% dropout per layer.

This paper has shown that Deep Neural Networks are not as competent as LSTM Networks to predict the Daily BSE Sensex. It is stated that both models can generalize new and volatile datasets, where the LSTM network outperformed DNN in making weekly predictions. It is also shown that LSTM networks are more competent in finding underlying trends and generating long term predictions on volatile stock datasets.

Returning to the context of this paper, *Shah et al.*, 2018 has reached a conclusion where LSTM, is more competent in generating predictions for volatile stock datasets. When Foreign Exchange Markets are compared to stock markets, it can be concluded that Forex Markets are more entropic. Having stated that LSTM networks are more competent than Deep Neural Nets to seek underlying trends in datasets, the Forex Market is still relatively hard to predict since there are a lot of factors which may influence its change, such as political factors. This introduces a research interest – how would the stock market's prediction RMSE metric differ to the RMSE metric for a Forex prediction.

Siami-Namini et al., 2018, which uses the statistical model, Autoregressive Integrated Moving Average (ARIMA) to compare with the LSTM architecture, has concluded, by using the S&P 500 index, Nikkei 225 Index, NASDAQ Composite Index, Hang Seng Index, Dow Jones Industrial Index and 6 other market prices, LSTM outperforms ARIMA by an average 85.92% reduction in root mean squared error (RMSE, which is modelled by $\sqrt{\frac{1}{n}[\sum(\hat{y}-y)^2]}$.

III: ALGORITHM DESIGN & EXPERIMENTAL HYPOTHESIS

To evaluate LSTM neural networks in Forex time-series forecasting, a series of experiments will be conducted using Keras as the Interface, with TensorFlow as the backend framework. This experiment is conducted under a Linux based GPU farm with 8 GeForce RTX 2080 Ti GPUs for accelerated machine training.

Section 1: Experimental Hypothesis

This experiment has been conducted under the following hypothesis, and aims to test it out using the algorithm constructed:

1. The Foreign Exchange Market, although consists of a higher level of randomness, would be able to provide a higher but similar RMSE accuracy statistic.

2. The model used, under the same hyperparameters, would be able to create a better prediction to the stock market than the Forex Market.

Section 2: Dataset and Data Preparation

For stock market section, I would use several composite stock indexes, such as S&P 500, NASDAQ Composite, FTSE 100, Hang Seng Index (HSI) and the Dow Jones Industrial (DJI) index to create a measurement of how accurate my network is to predict these markets.

As for the Foreign Exchange Market Dataset, the author has chosen the following indexes since the places associated with the Forex Market has the largest economical trading centres globally and there exists a large number of transparent trade records between the currencies, reducing the efficiency of the market. The indexes are listed below:

- United States Dollars to Hong Kong Dollars (USDHKD)
- United States Dollars to Euros (USDEUR)
- United States Dollars to Pound Sterling (USDGBP)
- Pound Sterling to Hong Kong Dollars (GBPHKD)
- Pound Sterling to Euros (GBPEUR)
- Euros to Hong Kong Dollars (EURHKD)

To keep the number of data points in the dataset listed above the same, 5 years of market data, from 1st January 2014 to 31st December 2019 within the 6 Forex Markets and 5 stock market indices is going to be used. This has been done such that the amount of data should only be a factor which affects the results obtained from the assessment minimally.

In this investigation, 4 weeks of trading days would be used to predict 1 week of trading days, *id est.* 20 data points will be used to predict 5 data points with a stride space of 1. Under this condition, the data is split into 70% training and 30% testing, which means that 70% of the data has been used to educate the neural network how it should generate a prediction for the 30%. The table below illustrates how the data is being split

and the amount of data after data processing. This has been done under empirical observation.

DATASET	OBSERVATIONS		
	TOTAL	TRAINING	TESTING
FTSE 100	1517	1037	432
HSI	1478	1010	420
NASDAQ	1510	1033	429
SP500			
DJI			
USDHKD	1565	1071	446
USDEUR			
USDGBP			
GBPHKD			
GBPEUR			
EURHKD			

Table 1: Data Splitting

A typical LSTM neural network architecture would prefer data which is within the range of 0 and 1 to maximize the learning efficiency whilst keeping statistical features within the dataset useful to learn. Hence, a linear-transformation scaling method named Min-Max Scaling would be used: this method compares the difference between the largest and the smallest value within the dataset and the difference between the largest and the current value in the dataset. This method preserves the general shape of the neural network.

To fairly compare the assessment metric between distinct datasets, where the range of raw data varies between from less than 1 to more than 20,000. I would measure root mean squared error between the scaled testing data the prediction under Min-Max Scaling.

There is a total of 4 columns which are not directly dependent on other data points in the dataset used for the assessment, labelled ‘Price’, ‘Open’, ‘High, ‘Low’. These data would be all used to generate a prediction. There are also 2 columns which are dependent on other columns, labelled ‘Date’ and ‘Change %’ which can be discarded since they would not contribute to the learning process of a neural network. The volume of the stock exchange market has not been used since there is not a volume for the Forex market.

Section 3: LSTM Neural Network Algorithm

To construct a suitable neural network to predict all 11 datasets, I would construct a neural network which consists of 4 LSTM hidden layers, connected by 2 Hidden Linear Layers. Under this configuration, I would use an LSTM network configuration of 32 cells, 64 cells, 256 cells and 512 cells from layer one to layer four respectively; followed by 1024 linear cells and 512 linear cells. The output layer would consist of 5 cells since I would want to predict 5 pieces of data, and the input layer would consist of 20 cells on a similar concept.

Between each layer, a dropout of 30% would be used to prevent overfitting and premature convergence. A total of 50 epochs would be used to allow the network to be trained properly to learn the most fundamental statistical representations and also prevent overfitting.

The ADAM optimizer, a first-order gradient optimization technique would be used instead of RMSprop and stochastic gradient descent since it has been proved effective in minimizing the validation loss, whilst the loss function would be the mean squared error function. The activation function across the entire network would be the linear activation function to increase the reliability and stabilize the backpropagation process since the gradient is constant.

The listing below shows the pseudocode of the algorithm.

Listing 1: Main Algorithm Pseudocode

```
Inputs: Market/Fourier Data
Outputs: RMSE of the Prediction
#Data Splitting
x_training = [[t1,t2,…,t20], [t2,t3,…,t21],…]
y_training = [[t21,t22,…,t25],[t22,t23,…,t26],…]
x_test = […,[tn-24,…,tn-6,tn-5]]
y_test = […,[tn-4,tn-3,…,tn]]

#Establish Neural Network
model = Sequential(
    Linear(20, activation='linear'),
    LSTM(32, return_sequences=True),
    Dropout(0.3),
    LSTM(64, return_sequences=True),
    Dropout(0.3),
    LSTM(256, return_sequences=True),
    Dropout(0.3),
    LSTM(512, return_sequences=True),
    Dropout(0.3),
    Linear(1024, activation='linear'),
    Linear(512, activation='linear')
    Linear(5)
)
model.compile(loss='mean_squared_loss', optimizer='adam')
model.fit(x_training, y_training, epoch=50)
yhat = model.predict(x_test)
return (sum((yhat-y_test)^2)/len(ytest))^0.5
```

IV: EXPERIMENTAL RESULTS

The results of this experiment have been reported in Table 2. This investigation has been repeated 10 times, and a mean and the standard deviation between the RMSE metric has been drawn and presented in the table below. The mean of the standard deviation has been calculated by measuring the standard deviation across all the experimental results for the type of dataset.

	RMSE MEAN		RMSE S.D.	
DATASET	SCALED	UNSCALED	SCALED	UNSCALED
SP500	0.02172	32.53999	0.000935	1.400366
NASDAQ	0.033401	167.8562	0.008737	43.90542
FTSE 100	0.044547	104.2605	0.00514	12.02977
HSI	0.042828	635.3364	0.010177	150.9729
DJI	0.026724	354.6938	0.006156	81.7113
STOCK MEAN	0.033844		0.011298	
USDHKD	0.026459	0.002651	0.003591	0.00036
USDEUR	0.044531	0.01091	0.004546	0.001114
USDGBP	0.036487	0.009071	0.006654	0.001654
GBPHKD	0.034865	0.138043	0.004725	0.018707
GBPEUR	0.035865	0.013173	0.001358	0.000499
EURHKD	0.054175	0.001712	0.007866	0.000249
FOREX MEAN	0.03873		0.010122	

Table 2: Experimental Results

It can be concluded that the measurement metrics from the experiments are very tightly compact by comparing the mean of the RMSE and the standard deviation at different experiments. This shows that this experiment should not consist of an overwhelming amount of random errors within the dataset; should there be an error, it would be a systemic error across all datasets, which proves that the conclusion drawn from this experiment should, therefore, be valid.

The RMSE metric between the Foreign Exchange Markets is very closely linked together, ranging from 0.034 to 0.054. The result that the RMSE metric for the market dataset between the US dollars and Hong Kong Dollars is lower than others should be an expected outcome since the US dollars and Hong Kong dollars have been pegged at a rate of 7.8 Hong Kong dollars to 1 US Dollar since 17th October 1983, reducing the extent of data fluctuation between the two datasets, and hence increasing the probability of statistical features being captured.

The RMSE metric within the stock dataset is lower than the Forex Market RMSE value and has a larger standard deviation across the stock dataset. There is a 14.437% increase in the mean of the RMSE metric between the stock and the Forex dataset. This is not an apparent change. There are no clear statements or indications that can be made regarding that the stock markets are easier to predict than the Forex Market.

V: DISCUSSION

The LSTM neural network architecture has shown a promising prediction to the Forex Market. The Foreign Exchange Market, although more stochastic, has only got an RMSE metric of 4.89×10^{-3} higher than the stock market. The neural network and optimizing algorithms used in this assessment have successfully converged the training loss and the validation loss to a smaller value with the datasets used in this paper.

Section 1: Hypothesis Tests and the KL Divergence

Deep learning architectures have been initiated with random values as weights and zeros as biases to prevent the model from being stuck at a local minimum and never reaching a global minimum since there isn't a momentum on initialization, with processes such as ADAM or stochastic gradient descent, the gradient of the loss function descends on a random batch selection. Since the model should, in theory, converge to a minimum, namely the global minimum, the distribution of the loss values should form a univariate Gaussian Distribution.

To compare the two distributions, a normal distribution hypothesis test can be performed. The null hypothesis is that the mean of the stock market RMSE metric is the same as the mean of the RMSE metric for the Foreign Exchange Market. Under this hypothesis, the significant value for the assessment has to be larger than 33.27% for the alternative hypothesis to be accepted. This means that there is a very high probability that a false positive conclusion has been reached.

To further compare the two distributions, a Kullback-Leibler Divergence (KL Divergence) can be used to compare the difference between the cross-entropy of the two distributions between the two metrics.

The KL divergence for any two continuous probability distributions is given as:

$$D_{KL}(\alpha||\beta) = \int_{-\infty}^{\infty} \alpha(x) \ln \frac{\alpha(x)}{\beta(x)} dx$$

For a univariate Gaussian Distribution, the KL divergence after evaluating the integral is:

$$D_{KL}(\alpha||\beta) = \log \frac{\sigma_\beta}{\sigma_\alpha} + \frac{\sigma_\alpha^2 + \left(\mu_\alpha - \mu_\beta\right)^2}{2\sigma_\beta^2} - 0.5$$

Now given the X_S is the distribution of the stock market and X_F is the distribution of the Forex Market.

$$X_S \sim N(0.033844, 0.011298^2)\ ;\ X_F \sim N(0.03873, 0.010122^2)$$

$$D_{KL}(X_F||X_S) \approx -0.3349$$

The small value for the KL Divergence signifies that the difference between the entropy of the two univariate Gaussian Distributions is very low. This means that for both tests, there is insufficient evidence to show that the difficulty to predict the Foreign

Exchange Market is higher than the stock market. Hence, for this assessment, the null hypothesis that the difficulty to predict the stock and the Forex is the same has been accepted.

Section 2: Potential Improvements

Under any hypothesis tests, the conclusion that 'there is insufficient evidence to reject the null hypothesis' does not imply that the null hypothesis is for definite true. Hence, to properly conclude, other approaches can be used to test whether this assessment's result is valid.

As a basic approach, this test can be redone with other hyperparameters or other neural network configurations. This draws to more results regarding whether Forex is harder to predict than stock, by using a majority voting system, should the result that the null hypothesis is consistently proven that it cannot be rejected, the conclusion that the Forex market and the stock market shares a similar difficulty to predict can, therefore, be implied. Further improvements such as using more samples to test each dataset can also be used, to draw a more significant mean and standard deviation.

An advanced approach to solving the problem is to use a non-linear transformation to the data, such that the data is reshaped into datapoints easier for the network to capture statistical dependencies by amending the statistical characteristics of the dataset. The network would then be able to generate a prediction to the market based on the non-linear transformation which can then be inverse transformed to compare with the result. Such functions include the hyperbolic tangent, sigmoid, *et cetera.*

Limitations to this approach include that the transformation has to be reversible, *id est.* a function such as a ReLU is not suitable; functions cannot be in the form of $f(x+h) = f(x)$, such as $\sin x$. This causes the data to be irreversible to its initial shape since there are an infinite amount of datapoints which would fit into a value whilst descaling, hence destroys the integrity of the data. The function is also required to converge to a number, or else it is going to diverge to a very large number for some input.

The data within the dataset has to be reshaped to smaller values, such as using a Min-Max Scaler, a linear transformation, before applying to a non-linear scaling function, such that the data in the dataset will be mapped to a suitable range of values.

Another optimizing algorithm, Adabound (Luo et al., 2019), which is a more effective optimizing algorithm than stochastic gradient descend and ADAM, can the acting compiling optimizer in this experiment after this technology mature to seek a better local minimum.

VI: CONCLUSION

This paper has first introduced the fundamental concepts of the economics of forecasting markets, such as Forex and stock markets. An introduction to Long Short-Term Memory has then been done, which is the desired algorithm to conduct this comparative study on the difference in the complexity to predict the Forex and stock market.

In this paper, 6 of the many most-traded Forex Markets globally, namely USD to HKD, USD to EUR, USD to GBP, GBP to HKD, GBP to EUR and EUR to HKD and 5 of many infamous composite stock markets, such as the Hang Seng Index (HSI), Financial Times Stock Exchange Index 100 (FTSE 100), Standard and Poor 500 (S&P 500), Dow Jones Industrial Average (DJI) and the NASDAQ composite index has been used to test whether the LSTM neural network can effectively capture statistical dependencies for Forex markets. To compare the difficulty to compare the two distinct markets, the author has established an LSTM neural network architecture and have compiled it with the ADAM optimizer and the mean squared error loss function. The training task has been repeated for 10 times for each of the dataset and the RMSE mean and the standard deviation has been obtained.

To compare the distributions of the prediction error metrics between the two markets, a Kullback-Leibler Divergence (KL Divergence), and a Normal Distribution Hypothesis testing has been used. The mean for the stock market RMSE metric is 14.437% lower than the RMSE metric for the Forex market. This comparison between the two markets has been done under the assumption that the distribution of the RMSE metric is a univariate Gaussian (Normal) distribution. The conclusion for this experiment is that the stock market is not as complex to predict than the stock market. This is drawn in consideration of the probability of accepting a false-positive result that the Forex market is harder to predict than the stock market.

Treating the null hypothesis as a true statement because it cannot be rejected is not a satisfactory result: the author has presented 3 methods for potential improvements to the experiment.

REFERENCES

CHAPPELOW, J. (2019). Market Efficiency. [online] Investopedia. Available at: https://www.investopedia.com/terms/m/marketefficiency.asp.

HOCHREITER, S. AND SCHMIDHUBER, J. (1997). Long Short-Term Memory, Neural Computation 9(8): 1735-1780, 1997. [online] Bioinf.jku.at. Available at: http://www.bioinf.jku.at/publications/older/2604.pdf

KINGMA, D. AND LEI BA, J., 2015. ADAM: A Method For Stochastic Optimization. [online] Arxiv.org. Available at: https://arxiv.org/pdf/1412.6980.pdf

LO, A., (2007). Part I Of Efficient Markets | Efficient Markets | Video Lectures And Slides | Finance Theory I | Sloan School Of Management | MIT OpenCourseWare. [online] Ocw.mit.edu. Available at: https://ocw.mit.edu/courses/sloan-school-of-management/15-401-finance-theory-i-fall-2008/video-lectures-and-slides/efficient-markets/part-i-of-efficient-markets/

LUO, L., XIONG, Y., LIU, Y. AND SUN, X. (2019). Adaptive Gradient Methods with Dynamic Bound of Learning Rate. [online] arXiv.org. Available at: https://arxiv.org/abs/1902.09843 [Accessed 3 Jan. 2020].

SIAMI NAMINI, SIMA & TAVAKOLI, NEDA & SIAMI NAMIN, AKBAR. (2018). A Comparison of ARIMA and LSTM in Forecasting Time Series. 1394-1401. 10.1109/ICMLA.2018.00227.

SHAH, DEV & CAMPBELL, WESLEY & ZULKERNINE, FARHANA. (2018). A Comparative Study of LSTM and DNN for Stock Market Forecasting. 10.1109/BigData.2018.8622462.